Files, editors and a project handover

Laboratory 02

Complete Tasks 2.01–2.06; then choose an extension if you have time. Keep your files between tasks. Use the terminal for file operations and the lecture or documentation when needed.

Working directory: start each task in `~/unix-lab02`. Paths below are relative to that directory unless a step changes your location.

Set up and edit

Task 2.01 A fresh workspace

1. Open a terminal. Check the host, username and current directory. Go home and create a new `unix-lab02` directory. Enter it. If it already exists, choose a fresh name and use it throughout this worksheet.
2. Create `input`, `notes`, `scratch` and `runs/run01`, including the missing parent directory. Inspect the layout with `tree` or `ls -R`.
3. Enter `input`, then reach `runs/run01` using a relative path. Check your location. Use `cd -` to return to `input`, then return to the workspace. Use Tab completion at least once.
4. Create an empty hidden file `notes/.session` and display its entry. Create `scratch/empty`, then remove that empty directory with `rmdir`.

Task 2.02 Prepare an input file with nano

Quick keys: `nano FILE` opens a file. `Ctrl+O`, then Enter saves; `Ctrl+X` exits; `Ctrl+W` searches; `Alt+U` undoes. Read the shortcuts at the bottom if your settings differ.

1. Use `nano` to create `input/settings.ini` with exactly these four lines. Finish the last line with Enter. Save, exit and display the file with `cat`.

```
steps = 1000
dt = 0.01
output_every = 100
seed = 42
```

2. Copy this file to `runs/run01/settings.ini`. Edit the copy in `nano`: change `steps` to `2000` and `output_every` to `250`. Use the editor's search to find one of these settings.
3. Make one deliberate typing mistake and undo it. Save and exit. Display both files: the original must still contain `1000` and `100`; the copy must contain `2000` and `250`.

Vim and symbolic links

Task 2.03 Edit, undo, save and leave Vim

Vim starts in **Normal mode**, where keys are commands. Press `i` to enter **Insert mode** and type text. Press `Esc` to return to Normal mode. Use arrow keys and Backspace while editing a value.

In Normal mode	Action
`/seed`, then Enter	Search for `seed`; `n` finds the next match.
`gg` / `G`	Go to the first / last line.
`dd`; `u`	Delete the current line; undo the last change.
`:wq`, then Enter	Save and quit.
`:q`, then Enter	Quit; refuse if there are unsaved changes.
`:q!`, then Enter	Discard *unsaved* changes and quit.

1. Create `runs/run02` and copy `runs/run01/settings.ini` into it. Open the new copy with `vim`.
2. Change `steps` to `3000`. Return to Normal mode, search for `seed`, then change its value to `99`. Leave the other two settings unchanged.
3. In Normal mode, delete one line with `dd`, then restore it with `u`. Check that all four lines remain. Save and quit.
4. Reopen the file, change `seed` to `7`, and try `:q`. Read the message, then leave with `:q!`. Display the file from the shell: the saved value must still be `99`. Does `:q!` undo a change already saved?

Task 2.04 A shortcut to the selected run

`ln -s TARGET LINK` creates a symbolic link storing a pathname; `readlink LINK` displays that pathname. A **relative target is interpreted from the directory containing the link**. The link is not a separate copy of the target's data.

1. From the workspace, create `runs/latest` pointing to `run02` with a relative target. Inspect it with `ls -ld runs/latest` and `readlink runs/latest`. Read `runs/latest/settings.ini`.
2. Go home and read the same file through `unix-lab02/runs/latest/settings.ini`. Why does the link still work? Return to the workspace.
3. Rename `runs/run02` to `runs/run02-old`. Try reading through `runs/latest` again. Inspect the link: did its stored target change? Restore the directory's original name and verify that the link works again.
4. Remove *only* `runs/latest` with `rm -i`; do not add a trailing `/`. Check that the target file still exists. Re-create the working link for later inspection.

Check your work and prepare a handover

Task 2.05 Compare files and check disk usage

Two additional tools: `wc -l FILE` counts newline characters (normally the number of text lines); `diff -u OLD NEW` compares two text files. Identical files produce no output from `diff`. In the differences, `-` marks old lines and `+` marks new lines, apart from the filename headers.

1. Check that each `settings.ini` file has four lines using `wc -l`. Compare `input/settings.ini` with the run01 copy using `diff -u`; then compare run01 with run02. Which two values changed in each comparison?
2. Run `du -sh .` and `df -h .`. Identify your project's disk usage and the free space on its filesystem. Which result covers other users' data too? Free filesystem space is not your personal quota.

Help: `du` measures disk usage of files/directories; `df` reports filesystem capacity. `-h` uses readable units; `-s` asks `du` for a total. See Lecture 02, *Filesystem hierarchy*.

Task 2.06 Independent task: deliver a ready-to-use package

Allow about **25 minutes**. Use the lecture, this worksheet and documentation. Choose the commands yourself. Show the instructor the finished result; you do not need to demonstrate every intermediate step.

1. Create `handover` with subdirectories `input`, `results` and `notes`. Copy the run02 settings into `handover/input/settings.ini`. Use **Vim** to change only `seed` to `123` in this new copy. Keep the earlier files in `input` and `runs` unchanged. Enter `handover`.
2. Save the first two lines of `input/settings.ini` to `results/first.txt` and the last two to `results/last.txt`. Combine these files, in order, into `results/rebuilt.ini` using one command.
3. Create a relative symbolic link `latest.ini` to `results/rebuilt.ini`. In `notes/run summary.txt`, record three lines: your login, the source path and what you changed. Give the source as `runs/run02/settings.ini`.
4. Return to the workspace. Rename `handover` to `handover-ready`. The link must work without being re-created. Check the final layout below.

```
handover-ready/
|-- input/settings.ini
|-- results/first.txt
|-- results/last.txt
|-- results/rebuilt.ini
|-- notes/run summary.txt
‘-- latest.ini -> results/rebuilt.ini
```

Show these results together:

- The package listing, including the link target, and your three-line note.
- A comparison proving that `input/settings.ini` and `results/rebuilt.ini` inside the package are identical, plus their line counts.
- From your home directory, display the file through `latest.ini`: it must contain `steps = 3000`, `dt = 0.01`, `output_every = 250` and `seed = 123`. The run02 source must still have `seed = 99`.

Optional extensions

These are independent. Work in the original workspace, outside `handover-ready`, so your completed package stays unchanged.

A Variables and a shortcut for this session

`name=value` assigns a variable (no spaces around `=`); `$name` reads its value. Quote values used as paths. `export` makes a variable available to new programs, and `printenv NAME` looks for it in the environment. `alias` defines a command shortcut in the current shell.

1. Set `labdir="$HOME/unix-lab02"`, adjusting the name if necessary. Display it with `echo "$labdir"`, then try `printenv labdir`. Run `export labdir` and repeat `printenv labdir`. What changed?
2. Start a child shell with `bash`. Display `labdir`, then assign `labdir=/tmp` in that child shell. Use `exit` once to return to the parent shell. Display the value again. Did the child’s change affect the parent?

3. Define `alias croot='cd "$labdir"'`. Go to `/tmp`, use `croot`, and check your location. Inspect the shortcut with `type croot`, then remove it with `unalias croot`. Remove the variable with `unset labdir`.

Keep these changes in the current session; no startup-file editing is needed.

B A hard link is not a backup

`ln FILE NAME` adds another name for the same file. `ls -li` shows inode numbers and hard-link counts; `stat FILE` shows metadata. Compare inode numbers only within the same filesystem.

1. Copy `runs/run01/settings.ini` to `scratch/original.ini`. Create `scratch/shared.ini` as a hard link to this file and `scratch/backup.ini` as a separate copy. Compare all three with `ls -li`.

2. Use `echo` and `>>` to append `# checked` to `scratch/shared.ini`. Read all three files. Which two changed, and why? Use redirection for this experiment: an editor may replace a file when saving.

3. Remove only `scratch/original.ini`. Read `scratch/shared.ini` and inspect its remaining link count. Use `stat` to find its size and modification time. Why would `shared.ini` be unsuitable as an independent backup?

C A little more Vim

Copy `runs/run02/settings.ini` to `scratch/vim-practice.ini` and open the copy in Vim. Practise these operations in **Normal mode**:

1. Move with `h`, `j`, `k`, `l`; then with `w` and `b`. Compare character movement with movement between word starts.

2. Search for `=` and use `n` and `N` to move between matches in opposite directions. Jump to the beginning and end with `gg` and `G`.

3. Delete a line with `dd`, undo with `u`, redo with `Ctrl+R`, then undo again. Save and quit. Compare the file with its source using `diff -u`.